

Forme normale di Backus

La definizione di linguaggi di programmazione consentono di caratterizzare un linguaggio di programmazione come e' insieme di tutte le stringhe derivabili dalle grammatiche

Per la definizione sintattica dei linguaggi di programmazione vengono adottate grammatiche non contestuali, di solito rappresentate mediante una notazione specifica, dette **FORMA NORMALE DI BACKUS (BNF)**

La BNF e' una notazione per grammatiche context free che e' "piu' espressiva" mediante delle notazioni che analiamo e definire

Queste notazioni sono "spiccatamente" ispirate alle espressioni regolari

1) Simboli non terminali sono sempre costituiti da stringhe che denotano delle categorie sintattiche racchiuse tra parentesi acute <...>

<espressione> chiamati identificatori

2) Il segno di derivazione ($\rightarrow$) viene sostituito dal simbolo ($::=$) per non confonderlo con i simboli $\rightarrow$, $=$, $::=$ che sono simboli terminali usati in veri linguaggi di programmazione

3) Le parentesi grasse $\{ \dots \}$ vengono usate per indicare e' iterazione illimitata ($0, 1, 2, \dots, n$, volte). Analogamente indichiamo con $\{ \dots \}^n$ l'iterazione per un numero di

valte almeno pari od n

PER ESEMPIO:

$A ::= BA \mid e$ può essere riscritta $A ::= \{B\}^n e$

usando quelle prod
lo possiamo scrivere
 ∞ volte

$BA \rightarrow BB A \rightarrow \dots \rightarrow BB \dots A \rightarrow BB \dots e$

oppure

$A ::= x \mid xy \mid xy y \mid xy y y y \mid xy y y y y y$ può essere riscritta
come $A ::= x \{y\}^5$

4) Le parentesi quadre vengono utilizzate per indicare
l'optionalità (possibile assenza di parte di una stringa)

$A ::= xy \mid y$ può essere riscritta $A ::= [x]y$
le x potrebbe
non esserci

5) Le parentesi tonde (...) vengono utilizzate per evitare
la sottoespressione, vale a dire le meno in comune di una
sottoespressione

Per esempio

$A ::= x u \mid x v \mid x y$ può essere riscritta come

$A ::= x(u \mid v \mid y)$

FORMA NORMALE DI CHOMSKY

Definizione:

Sia $G = \langle V_+, V_N, P, S \rangle$ Context free. La grammatica G è detta in forma normale di Chomsky o CNF (Normal Form (CNF))

Se tutte le regole di produzione di G sono delle forme:

$$A \rightarrow BC \quad \text{con } A, B, C \in V_N$$

$$\text{oppure } A \rightarrow a \quad \text{con } a \in V_+$$

Proposizione

Dato una qualunque grammatica context free di tipo 2 ne esiste una equivalente in CNF del tipo

$$S \rightarrow aSb \mid \epsilon$$

e questo si dimostra tramite una dimostrazione costruttiva

Esempio

$A \rightarrow aXbY$ grammatica di tipo 2 (in questo momento non è in forma normale)

Allora eseguo delle trasformazioni

$$\begin{cases} A \rightarrow BXY \\ B \rightarrow a \\ C \rightarrow b \end{cases} \quad 1^\circ \text{ trasformazione}$$

$$\begin{cases} A \rightarrow BZ_1 \\ Z_1 \rightarrow XZ_2 \\ Z_2 \rightarrow cY \end{cases} \quad 2^\circ \text{ trasformazione}$$

Le regole di produzioni aumentano ma il risultato è uguale e prima

Significa che abbiamo definito una grammatica equivalente a quella di tipo 2 definite in forma normale

FORMA NORMALE DI GREIBACH

2 definite: $A \rightarrow \alpha$ $A \in V_N$ $\alpha \in V_T$
 $\alpha \in V_N^*$ insieme della stringhe non
terminanti

Le produzioni sono del tipo

$$S \xrightarrow[G]{} \alpha \beta \quad \alpha \in V_T^* \quad \beta \in V_N^*$$

Nell'automato a pila un po' equivale a far camminare la stringa α e scrivere sul nastro β . Anche in questo caso come per le grammatiche di tipo 2 ne esiste una in forma normale di Greibach. Si nota che in forma normale non esiste derivazione sinistra e quindi possiamo scrivere che

$$A \xrightarrow[G]{}^* \alpha \beta$$

Esempio di Backus

$$V_T = \{e, b, \dots, z, A, B, \dots, Z, 0, \dots, 9\}$$

$$\langle \text{identificatore} \rangle ::= \langle \text{alfabetico} \rangle \{ \langle \text{alfanumerico} \rangle \}^{n-1}$$

$$\langle \text{alfabetico} \rangle ::= e | b | \dots | z | A | B | \dots | Z$$

$$\langle \text{alfanumerico} \rangle ::= \langle \text{alfabetico} \rangle | \langle \text{cifre} \rangle$$

$$\langle \text{cifre} \rangle ::= 0 | 1 | \dots | 9$$

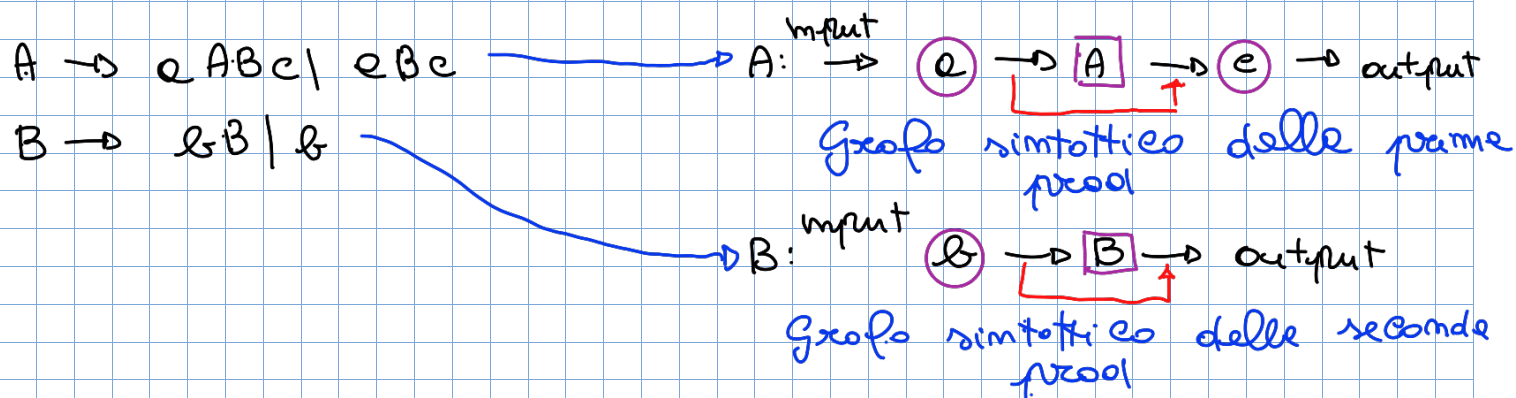
Capitolo 2 Linguaggi formali esempio 2.24
esempio 2.25 BNF per il Pascal

Un'altra notazione usata per descrivere i linguaggi di programmazione è costituita dai diagrammi sintattici:

$\Downarrow$
li possiamo rappresentare
come grafici orientati

Con un ingresso, un'uscita e i cui nodi sono collegati tra loro, tramite archi orientati ed etichettati con uno o più caratteri terminali e non terminali. I caratteri terminali vengono nelle forme a e b e i caratteri non terminali sono inclusi all'interno di un quadrato

Definiamo il digramma sintattico delle seguenti grammatiche



Esercizi sulle grammatiche

1) Verificare che la grammatica è regolare

G:

$$S \rightarrow aS \mid bA$$

$$A \rightarrow bA \mid \epsilon$$

ogni produzione

$$S \rightarrow aS \quad \checkmark$$

$$S \rightarrow bA \quad \checkmark$$

$$A \rightarrow bA \quad \checkmark$$

$$A \rightarrow \epsilon \quad \text{è un non terminale di fine catena}$$

Per essere regolare

$$A \rightarrow aB$$

verifichiamo queste condizioni

1) Queste grammatiche è regolare destra

2) Generazione del linguaggio da una grammatica

$$G: S \rightarrow eS \mid bS \mid \epsilon$$

Quale linguaggio genera G ?

$$S \rightarrow eS \rightarrow ebS \rightarrow ebbS \\ \searrow \rightarrow ebeS$$

ogni produzione una e o
una b e richiama S
oppure termina con ϵ

2) Generare tutte le stringhe (anche le vuote) su $\Sigma = \{e, b\}$
e quindi il linguaggio generato da G è $\{e, b\}^*$

3) Classificazione delle grammatiche

$$G = \langle \{e, b\}, \{S\}, P, S \rangle$$

$$S \rightarrow eSb$$

$$S \rightarrow \epsilon$$

partiamo da un non terminale e
produciamo terminali o non hanno
terminali

3) $\bar{E}$ di tipo 1

Capire lineare
destra / sinistra

4) Generare il linguaggio dell'esercizio 3

$$S \rightarrow \epsilon$$

$$\searrow \rightarrow eSb \rightarrow eeSbb \rightarrow eeeSbbb$$

$$L(G) = \{e^n b^n \mid n \geq 0\}$$

5) Classificazione

$$S \rightarrow eA$$

$$A \rightarrow eB$$

$$B \rightarrow b$$

Sono lineari destra le produzioni del tipo:

$$X \rightarrow wy \text{ oppure } X \rightarrow w$$

5) $\bar{E}$ di tipo 3

$$S \rightarrow eA \rightarrow eeB \rightarrow eeb$$

$$L(\bar{E}) = \{e^n b\}$$

6) Classificazione

$$eS \rightarrow Sb$$

$$S \rightarrow eb$$

$$eS \rightarrow eb$$

$$\rightarrow eeb$$

6) tipo 1

$$4) S \rightarrow \epsilon$$

$$SAB \rightarrow eSb$$

7) tipo 0

8) Costruire il linguaggio $L = \{e^n \mid n \geq 1\}$

$$S \rightarrow eS \mid e$$

8) Soluzione

PS: $\bar{E}$ di tipo 3

